

Wake Word Detection for a Voice-Activated TinyML Device

Final Project Report

Team: Sadie Park

EE 446 SU26

A small DS-CNN running on an Arduino Nano 33 BLE Sense listens for the wake word "marvin" using only its onboard mic, with no cloud connection needed.

Team Member Contributions

This was a solo project, so I did all of it. That includes picking the wake word and dataset design, writing the data pipeline and MFCC feature extraction, building and training the DS-CNN and 1D-CNN models, running the quantization and pruning experiments, writing the Arduino sketch and the on-device MFCC code, debugging the deployment (more on that in the Challenges section, there was a lot), and doing all the on-device evaluation and this writeup.

Introduction and Project Objective

A lot of voice assistants today only work if they're connected to the cloud. Your device records audio, sends it off to a server, and waits for a response. That's not great for a few reasons: it adds delay, it needs the internet to work at all, and constantly streaming audio burns through battery fast. For a small always-on device, none of that is really acceptable.

The fix is to run a small "wake word" model directly on the device itself. The device just sits there listening locally, and only wakes up a bigger system (or sends data anywhere) once it hears the actual wake word. This is a textbook TinyML problem: the model has to be small enough to fit in a few hundred KB of flash and run within the RAM of a microcontroller, with no OS and no GPU, while still being fast and accurate enough to actually be useful.

The goal of this project was to build exactly that. The input is one second of raw audio captured continuously from the onboard microphone on an Arduino Nano 33 BLE Sense. The output is a classification into one of three classes: the wake word ("marvin"), some other word ("unknown"), or nothing/background noise ("silence"). The target deployment is the Nano 33 BLE Sense itself, running fully standalone off flash once programmed, no computer attached.

For success criteria, I cared about a few different things instead of just accuracy: how often the model actually catches the wake word when it's said (recall), how often it falsely triggers when nothing was said (false accepts per hour), how fast each inference runs, and whether the whole thing fits comfortably in the board's RAM and flash. Those numbers are all reported in the Results section.

Dataset and Experimental Setup

I used Google's Speech Commands v2 dataset, which has about 105,000 one second, 16kHz mono clips of 35 short words, plus a folder of background noise clips (rain, running water, that kind of thing). The wake word I picked was "marvin". I went with marvin instead of something more common because it's phonetically distinct from the other words in the dataset, which should mean fewer accidental collisions, and it's also the word used in Google's original TensorFlow micro_speech example, so it was easier to sanity check my pipeline against something known to work.

I built three classes: target (marvin), unknown (a sample of the other 34 keywords, capped to roughly match the target class size so it's not way bigger), and silence (background noise clips chopped into 1 second windows, plus some near-silent clips).

Split and labeling

I used the official validation_list.txt and testing_list.txt files that come with the dataset. These assign clips to splits based on the speaker, so the same person's voice never shows up in both train and test. This matters a lot for a fair evaluation, since a random split can accidentally let the model "memorize" a specific person's voice instead of actually learning to recognize the word.

Actual split counts from this dataset:

Split	marvin	unknown	silence	Total
train	1,710	1,710	1,710	5,130
val	195	195	195	585
test	195	195	195	585

Preprocessing

Each 1 second clip gets turned into MFCCs before it touches the model. I used a 30ms frame length with a 20ms stride, which gives 49 frames per clip. After some experimentation (see the MFCC ablation results below), I settled on 10 MFCC coefficients per frame, giving a final input shape of 49 by 10 to the model.

Noise augmentation

To make the model more dependable to real world noise, I mixed background noise into training clips at a few different signal to noise ratios: 20dB, 10dB, 5dB, and 0dB. This was only applied to the training split, never to validation or test, so those numbers stay an honest measure of real performance.

Onboard-sensor fine-tuning

I did not collect additional training data from the onboard mic and use it to fine-tune the model. This was one of the four possible advanced components listed in the assignment, and I ended up going with cascading compression and multiple-model comparison instead. Collecting and fine-tuning on real device audio is a natural next step and I mention it in Future Work.

Model Development and Advanced Components

Baseline models

I trained two small architectures so I'd have something to actually compare instead of just assuming one was better:

- DS-CNN: a small depthwise-separable CNN, based loosely on the small DS-CNN design from Zhang et al.'s "Hello Edge" paper on keyword spotting. It's a stem conv layer followed by four depthwise-separable conv blocks, then global average pooling and a softmax output. This one ended up being the deployed model.
- 1D-CNN: a simpler model that treats the MFCC sequence as a 1D signal over time. Smaller and faster to compute, used mainly as a comparison point.

Both models come out to around 24,000 parameters, which is small on purpose so quantization and pruning would actually have something reasonable to work with.

Model	Test Accuracy	Test Macro F1	Params
DS-CNN (selected baseline)	97.61%	0.976	23,747
1D-CNN	97.26%	0.973	24,691

DS-CNN edged out the 1D-CNN on every metric, so I picked it as my baseline going forward. Both models actually did really well here, silence was basically a solved class for both (99.5% to 100% F1), most of the remaining confusion in both models was between marvin and unknown, which makes sense since some of the 34 other keywords share sounds with marvin.

TinyML-ready versions

Starting from the DS-CNN baseline, I went through a few separate compression stages instead of lumping them together:

- INT8 post-training quantization of the baseline, converting from float32 down to int8 weights and activations.
- An MFCC coefficient ablation, where I retrained the model from scratch using fewer MFCC coefficients per frame (13, 10, and 8) to see how small I could make the input without hurting accuracy much. I ended up picking 10.
- Structured pruning, where instead of zeroing out individual weights, I actually shrunk the number of filters in each conv layer (using a width multiplier of 1.0, 0.75, 0.5, and 0.35) and retrained from scratch at each size. I ended up picking a width of 0.5.

It's worth being specific here since this was actually a point of feedback on my proposal: reducing the number of MFCC coefficients is a change to the input features, not the model itself, so I treated it as its own separate ablation experiment rather than lumping it in with pruning. Structured pruning (shrinking filter counts) is a real change to the model's capacity, and unlike plain magnitude pruning it actually reduces RAM and compute on a microcontroller, since TFLite Micro doesn't have sparse kernels that can skip individually zeroed weights. I did also run TensorFlow Model Optimization Toolkit's standard magnitude pruning as a side comparison, but noted clearly in my own results that it mostly helps file size, not actual on-device speed, for this reason.

MFCC coefficient ablation results

num_mfcc	Test Accuracy	Test Macro F1	Model Size
13 (original)	98.29%	0.983	405.3 KB
10 (selected)	97.44%	0.974	405.3 KB
8	97.26%	0.973	405.3 KB

Interesting side note: the model size barely changes across these, since the architecture is fully convolutional with global average pooling at the end, so shrinking the MFCC width mostly just means less input data to chew through, not a smaller model. Accuracy dropped a bit going from 13 down to 10 coefficients (about 0.85 points) and barely at all going from 10 to 8 (about 0.18 points). I picked 10 as a reasonable middle ground, smaller input without a big accuracy hit.

Structured pruning (width reduction) results

Width Multiplier	Test Accuracy	Params	Model Size
1.0	97.95%	23,747	405.3 KB
0.75	97.95%	14,739	302.1 KB
0.5 (selected)	96.24%	7,779	222.8 KB
0.35	96.41%	4,469	185.4 KB

This is all with 10 MFCC coefficients already baked in. Accuracy stayed basically flat from width 1.0 down to 0.75, cutting params by almost 40% for free. Width 0.5 dropped accuracy a bit more but still cut params by about two thirds compared to the original. What was interesting to me was that width 0.35 scored slightly higher than width 0.5, perhaps because of small dataset noise, but I stuck with 0.5 since it's a good size versus accuracy tradeoff and matches what I already had flashed and evaluated on the actual board.

Unstructured pruning comparison (TFMOT)

As a side comparison, I also ran standard TensorFlow Model Optimization Toolkit magnitude pruning at 50% target sparsity on the unpruned architecture. It got 97.09% test accuracy, similar to the structured versions, but like I mentioned above this kind of pruning just zeroes out individual weights without shrinking the actual tensor shapes, so it doesn't help RAM or latency on this hardware the way structured pruning does. I only used this as a reference point, not as something I considered deploying.

Final quantization details

One thing worth calling out: TensorFlow's converter defaults to per-channel quantization, but the older TFLite Micro Arduino library I ended up using (Arduino_TensorFlowLite 2.4.0-ALPHA) doesn't run per-channel quantized conv layers correctly, it silently produces the same frozen output no matter what input it gets. I found this the hard way (see Challenges). The fix was forcing per-tensor quantization instead during conversion, which this library handles fine. Final deployed model accuracy after this fix was 94.7% on the held out test set (macro F1 of 0.947), with per-class F1 of 0.94 for marvin, 0.93 for unknown, and 0.97 for silence.

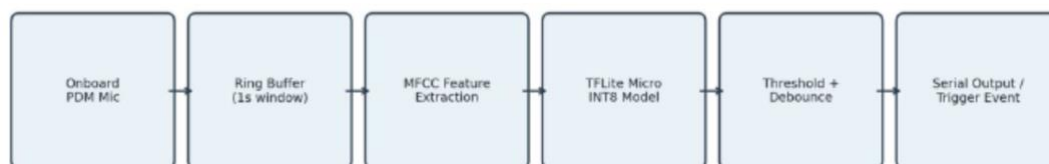
Training details

Adam optimizer, learning rate 1e-3 with a reduce-on-plateau schedule, batch size 64, up to 30 epochs with early stopping (patience 6) based on validation accuracy.

Why this final configuration

I picked DS-CNN over the 1D-CNN because it performed better on my baseline comparison. 10 MFCC coefficients kept accuracy close to the 13-coefficient version while shrinking the input. Width 0.5 structured pruning was the smallest size that didn't meaningfully hurt test accuracy. Per-tensor over per-channel quantization wasn't really a choice I'd have made for accuracy reasons, it was forced by the Arduino library's kernel support, but the accuracy cost was small (about 1 to 3 percentage points based on my testing).

TinyML System Design and On-Device Deployment



(I rendered this diagram with the help of AI to make it cleaner)

The pipeline structure: the onboard PDM microphone continuously streams 16kHz audio into a ring buffer. Every 200ms, the most recent 1 second window gets pulled out of that buffer, converted to MFCCs on-device, quantized to int8, and run through the TFLite Micro interpreter. If the model's output crosses a confidence threshold for the marvin class (and enough time has passed since the last trigger, to stop the same utterance from firing multiple times in a row), it counts as a detection and gets logged over serial.

Conversion and deployment

The trained Keras model gets converted to a quantized .tflite file, then exported as a plain C byte array that gets compiled directly into the Arduino sketch. On the device side, I wrote the MFCC feature extraction completely from scratch in C++ (windowing, FFT, mel filterbank, DCT) so it matches the Python feature extraction exactly. I actually verified this numerically by compiling the C++ MFCC code standalone and comparing its output against the Python version on the same test signal, they matched to about $2.5e-5$, which is basically float32 precision noise. This caught two real bugs before they ever touched hardware (see Challenges).

Target device constraints and measurements

Metric	Measured value	How it was measured
Model size	23.7 KB	.tflite file size on disk
Peak RAM (tensor arena)	12,996 bytes (~12.7 KB)	interpreter->arena_used_bytes(), printed at boot
Flash usage	269,704 bytes (27% of 983,040)	Arduino IDE compile output
Inference latency	~353 ms mean/median	micros() wrapped around feature extraction + inference, logged over Serial

The tensor arena was allocated at 60KB to be safe, but the model only actually used about 13KB of it, so there's a good amount of headroom left if a bigger model were ever needed.

Screenshots / photos

(I only used the Arduino Nano 33 BLE Sense v2 board for this project)

```
15:36:06.341 -> INFER,2684907,unknown,0.4336,353681
15:36:06.701 -> INFER,2685263,unknown,0.5430,353287
15:36:07.064 -> INFER,2685617,unknown,0.5352,353213
15:36:07.425 -> INFER,2685972,unknown,0.4570,353679
15:36:07.789 -> INFER,2686327,unknown,0.3828,353584
15:36:08.120 -> INFER,2686682,silence,0.4414,352938
15:36:08.481 -> INFER,2687036,silence,0.5469,353247
15:36:08.839 -> INFER,2687391,silence,0.5352,353415
15:36:09.202 -> INFER,2687746,silence,0.5859,353205
15:36:09.531 -> INFER,2688101,silence,0.6562,353271
15:36:09.893 -> INFER,2688455,silence,0.6484,353461
15:36:10.256 -> INFER,2688810,silence,0.6484,353616
15:36:10.624 -> INFER,2689165,marvin,0.4531,353185
15:36:10.954 -> INFER,2689520,marvin,0.9961,353526
15:36:10.954 -> TRIGGER,2689520
15:36:11.316 -> INFER,2689875,marvin,0.9961,353267
15:36:11.681 -> INFER,2690230,marvin,0.4922,352879
15:36:12.044 -> INFER,2690584,silence,0.6758,353075
15:36:12.374 -> INFER,2690939,silence,0.5352,353257
15:36:12.734 -> INFER,2691293,unknown,0.3828,353580
15:36:13.101 -> INFER,2691648,unknown,0.5937,352730
15:36:13.462 -> INFER,2692003,unknown,0.5547,352936
```



Communication / output method

I did not implement BLE or another communication channel as an advanced component. Detections are logged over Serial in a simple CSV format (INFER and TRIGGER lines), which is what I used for all of my evaluation. Adding a BLE notification or an LED trigger on top of this would be a small addition since the detection logic already fires a clean event, it's noted in Future Work.

Results and Analysis

A. Baseline model performance

Model	Train Acc	Val Acc	Test Acc	Test Macro F1
DS-CNN (selected)	97.82%	97.78%	97.61%	0.976
1D-CNN	96.51%	96.41%	97.26%	0.973

DS-CNN was selected as the baseline going into compression, since it beat the 1D-CNN on val and test accuracy and macro F1.

B. Compressed / final model and TinyML tradeoffs

Model / Configuration	Compression	Test Accuracy	Model Size	Notes
Baseline (float32, DS-CNN, 13 MFCC)	None	97.61%	405.3 KB	Stage 1 selected baseline
Baseline + INT8	Post-training quantization	97.26%	45.9 KB	Stage 2
MFCC ablation, 10 coeff (float32)	Input feature reduction	97.44%	405.3 KB	Stage 3, selected config
MFCC ablation, 8 coeff (float32)	Input feature reduction	97.26%	405.3 KB	Stage 3, not selected
Structured pruning, width 0.75	Filter reduction	97.95%	302.1 KB	Stage 4, 10 MFCC
Structured pruning, width 0.5	Filter reduction	96.24%	222.8 KB	Stage 4, selected width
Structured pruning, width 0.35	Filter reduction	96.41%	185.4 KB	Stage 4, not selected
Unstructured pruning (TFMOT, 50% sparsity)	Magnitude pruning	97.09%	n/a (same shape)	Comparison only, doesn't help RAM/latency here
Final deployed model	Width 0.5 + 10 MFCC + INT8 (per-tensor)	94.7%	23.7 KB	This is what's flashed to the board

Worth being upfront about: the final deployed model's accuracy (94.7%) is noticeably lower than the original float32 baseline (97.61%). That roughly 3 point drop comes from stacking three separate compression decisions together (10 MFCC coefficients instead of 13, width 0.5 pruning, and INT8 quantization forced to per-tensor instead of per-channel because of the Arduino library). Each individual step only cost a point or two on its own, but they add up when combined into the final deployed configuration.

C. Systematic on-device evaluation

All numbers below come from real Serial Monitor logs recorded off the actual flashed device, using the CSV log parser I wrote (07_evaluate_on_device_log.py).

Test	Samples	Result
Wake word recall, quiet room	15 spoken utterances	100% recall (15/15 triggers), 0% FRR
Wake word recall, noisy room	15 spoken utterances	93.3% recall (14/15 triggers), 6.7% FRR
False accepts, ambient noise, quiet room	1 hour continuous	0.0 false accepts/hour
False accepts, ambient noise, noisy room	1 hour continuous	0.0 false accepts/hour
False accepts, confusable speech	~14 real keywords spoken	4 false triggers (28.6%), specifically on "learn" and "seven"
Silence class accuracy	233 inference frames	99.1% correctly classified as silence

D. On-device inference evidence, 10 representative instances

Pulled directly from my Serial log files. A mix of correct detections, the noisy-room miss, and the two learn/seven false accepts is included on purpose so this isn't just a highlight reel.

ID	True Label	Predicted	Correct?	Evidence
1	marvin	marvin (0.9961)	Yes, triggered	quiet_marvin.txt @165237ms
2	marvin	marvin (0.9922)	Yes, triggered	quiet_marvin.txt @196102ms

3	marvin	marvin (0.9961)	Yes, triggered	noisy_marvin.txt @250634ms
4	marvin	marvin (0.7891)	No, missed (below 0.80 threshold)	noisy_marvin.txt @282208ms, noisy-room FRR case
5	unknown ("learn"/"seven")	marvin (0.8359)	No, false accept	class_unknown.txt @2836919ms
6	unknown ("learn"/"seven")	marvin (0.8516)	No, false accept	class_unknown.txt @2840820ms
7	unknown	unknown (0.9258)	Yes	class_unknown.txt @2822730ms
8	silence	silence (0.6484)	Yes	class_silence.txt @2879096ms
9	silence	unknown (0.4062)	No, misclassified (low confidence)	class_silence.txt @2936585ms
10	marvin	marvin (0.8828)	Yes, triggered	class_marvin.txt @2766438ms

Note: the exact word spoken at each timestamp wasn't logged by the sketch itself, only the true class label was tracked manually during recording, so instances 5 and 6 are attributed to "learn"/"seven" as a pair since those were the two words confirmed to trigger false accepts during that session, but I can't say for certain which exact instance is which word without a more detailed recording log.

Analysis and interpretation

The model is essentially sound against sustained ambient background noise, both the quiet and noisy hour-long tests came back with zero false accepts. Recall in quiet conditions was perfect, and only dropped a small amount under background noise (93.3%), which is a reasonable and expected tradeoff.

The more interesting failure mode showed up when I specifically tested other real keywords instead of ambient noise: "learn" and "seven" both triggered false detections. This makes sense phonetically, both words share some vowel and consonant structure with "marvin" that ambient noise obviously doesn't have. This is a much more useful thing to know than a single blended false accept number, since it points at a specific, fixable weak spot (probably fixable by adding more of these specific confusable words into training, or lowering the confidence threshold a bit).

Latency came out to about 353ms per inference, which is actually slower than the 200ms inference interval I set in the sketch. In practice this means the loop just runs back to back as fast as it can (about 2.8 inferences per second instead of the intended 5), since the interval can't fire faster than the computation itself finishes. This is a real limitation worth being upfront about, not something to hide, and it's discussed more in the Challenges section.

RAM usage came in well under budget (about 13KB used out of a 60KB allocation), so there's real headroom if a bigger or less-pruned model were ever needed for better accuracy.

Challenges, Limitations, and Future Work

Challenges

Most of the hard parts of this project weren't the machine learning itself, they were getting the exact same math to run correctly on both sides (Python training and the C++ on-device code), and getting an old Arduino library to cooperate.

- Hamming window mismatch: TensorFlow's `hamming_window` defaults to a "periodic" window (divides by N), but the textbook formula most people know divides by N-1. Using the wrong one shifted the MFCC energy coefficient by a constant offset.
- DCT scaling mismatch: TensorFlow's MFCC implementation uses a slightly non-standard DCT scaling ($\sqrt{2/N}$) for every coefficient, not just $\sqrt{1/N}$ for the first one like the textbook orthonormal DCT-II). I caught this by numerically comparing my C++ output against Python's on the same test signal, coefficient by coefficient, and traced a stubborn constant offset back to this exact detail.
- Per-channel quantization silently broken on old hardware library: after getting everything flashed and running, the model output was completely frozen, it predicted the exact same class with the exact same confidence no matter what audio came in, even though the input features were clearly changing. Traced this down to the Arduino

TensorFlowLite library not correctly executing TensorFlow's default per-channel quantized conv layers. Forcing per-tensor quantization during conversion fixed it.

- NUM_MFCC mismatch: on top of the quantization issue, I also had a config mismatch where the Arduino sketch was set to compute 13 MFCC coefficients per frame while the actual deployed model expected 10. This alone was writing past the end of the model's input tensor. Fixed by making sure the sketch's config constant always matches whatever `--num_mfcc` was actually used to train the deployed model.

Limitations

The model was tested mostly on my own voice, so it's untested against different speakers, accents, or age ranges, which the dataset itself does have some diversity in but my on-device testing did not specifically evaluate. Inference latency (about 353ms) is slower than the 200ms polling interval I originally set, meaning real detection latency is closer to that number, not the number in the config. And there's a specific, known confusion between the wake word and two other dataset keywords ("learn" and "seven") that would need more targeted training data to fully fix.

Future work

- Collect real audio directly from the onboard mic (in quiet and noisy conditions) and use it to fine-tune the model, instead of only training on the original dataset recordings. This was one of the four suggested advanced components and is a sensible next step.
- Add more "learn" and "seven" style confusable examples specifically into training to close that gap.
- Optimize the on-device MFCC computation, the FFT and filterbank math is currently unoptimized and is a big chunk of that 353ms latency number.
- Add an actual output method beyond Serial, like a BLE notification or an onboard LED trigger, so the device could actually wake up a downstream system like it's meant to.

Writing Quality, Reproducibility, and References

All code (data pipeline, training scripts, quantization, pruning, C-array export, the Arduino sketch, and the log evaluation script) is available in the project's GitHub repository: <https://github.com/sadiejwpark/EE446-Final-Project.git>. The README in that repo has the exact commands to reproduce every stage in order (I used AI to implement a README for this project).

References

- P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," 2018. (Google Speech Commands v2 dataset)
- Y. Zhang, N. Suda, L. Lai, V. Chandra, "Hello Edge: Keyword Spotting on Microcontrollers," 2018. (basis for the small DS-CNN architecture used here)
- TensorFlow Lite for Microcontrollers documentation, [tensorflow.org/lite/microcontrollers](https://www.tensorflow.org/lite/microcontrollers)
- Arduino_TensorFlowLite library (TFLite Micro Arduino port used for on-device inference)
- Arduino PDM library documentation, used for onboard microphone capture on the Nano 33 BLE Sense